Goxpyriment: A Go Framework for Behavioral and Cognitive Experiments

Christophe Pallier, Julie Bonnaire, and Marie-France Fourcade
Cognitive Neuroimaging Unit
CNRS INSERM CEA

Abstract

We introduce `goxpyriment`, a new open-source software framework for programming behavioral and cognitive experiments using the Go programming language. The library is designed to address some limitations of existing Python-based experiment tools, particularly the runtime environment complexity that frequently complicates deployment across laboratories. Because Go is a compiled language that can natively embed assets (e.g., graphics, audio files, and stimulus lists), `goxpyriment` compiles entire experiments into single, self-contained executable binaries with zero runtime dependencies. This drastically simplifies distribution to collaborators and testing computers. `goxpyriment` prioritizes timing reliability. Go's garbage collector can be disabled, highly-reducing the probability of unpredicable pauses, latencies that can ruin the timing of the experiment. Keyboard and mouse events' timestamps at the operating system level are available, allowing one to avoid continuous pooling to get precise reaction times. The programming interface, inspired by Expyriment (Krause & Lindemann, 2014), was designed be human friendly. The library includes an array of visual stimuli (text, shapes, images, Gabor patches, motion clouds, ...) and audio capabilities (WAV playback and tone generation). A set of 35 classic psychology experiments implemented in goexpriment are provided that promote not only learning by humans but also boost the ability of modern AI-assisted coding tools to program experiments. Finally, a migration guide from other frameworks is provided. The framework is released under the GNU General Public License v3 and is freely available at https://github.com/chrplr/goxpyriment.

Keywords: framework, library, stimulus presentation, timing, behavioral experiments, psychophysics, psychology

---

Christophe Pallier https://orcid.org/0000-0002-3324-666X Correspondence should be addressed to C. Pallier, Cognitive Neuroimaging Unit, Neurospin, Gif-sur-Yvette, F-91191, France. E-mail: christophe@pallier.org. I would like to thanks Luca Bonatti and Raphael Bordas for alpha testing this software. The authors have no conflicts of interest to disclose.

Introduction

Experiment-control software occupies a central position in behavioral and cognitive neuroscience: the temporal precision of stimulus delivery and response recording, the ergonomics of experiment programming, and the ease of deploying experiments to laboratory hardware each affect the quality and reproducibility of empirical data. Over the past two decades, Python has become the dominant language for experiment programming, primarily through two libraries: Expyriment (Krause & Lindemann, 2014) and PsychoPy (Peirce et al., 2019). Before Python, MATLAB with the Psychophysics Toolbox (Brainard, 1997; Kleiner et al., 2007) was the de facto standard in many laboratories. More recently, web-based frameworks such as jsPsych (de Leeuw, 2015; de Leeuw et al., 2023) and lab.js (Henninger et al., 2022) have enabled large-scale online data collection.

Each class of tool involves tradeoffs. MATLAB, extended with specialized C/C++ libraries (e.g. the Psychtoolbox), can offer good timing on well-configured hardware. Yet Psychtoolbox scripts written on Windows often do not work out of the box on Linux (and vice-versa). MATLAB can also quite expensive if you one obtain academic licenses (as is the case in research institutions that do not deliver diplomas). Python tools provide a more open, flexible programming environment with a large scientific ecosystem. Python-based experiments can achieve high-precision timing, for example by relying on the psychtoolbox python module which provides pieces of the Psychtoolbox-3 (See https://pypi.org/project/psychtoolbox/). However, the use of Python introduces practical difficulties. First, deploying an experiment to a computer that does not already have a compatible Python environment requires package management (conda, pip, venv) and can be a source of breakage across operating-system versions. For example, installing the psychopy python module under Linux often crashes while attempting to compile the wxPython package when there is no precompiled wheel. Second, Python's garbage collector can introduce unpredictable pause times that corrupt the timing of stimulus presentation and response recording during the critical trial loop. Web-based tools trade timing precision for accessibility: browser-mediated stimulus presentation is subject to event-loop latency that make sub-frame timing difficult or impossible (Anwyl-Irvine et al., 2021; Bridges et al., 2020).

The Go programming language (Donovan & Kernighan, 2016) offers a different perspective. Designed from the ground up for simplicity, Go has fewer moving parts than Python; as a consequence, the code is often described as 'boring' but highly predictable, as the language structure typically enforces only one way to solve a given problem. In addition, Go is strongly typed, and encourage error checking. This translate into less run-time debugging. Another interesting feature of Go is that it (cross-)compiles to native machine code for each target platform and can produce self-contained binaries, which can embed the stimuli, expeiment lists, and this have no runtime dependencies. Its garbage collector is concurrent and can be suspended for the duration of a timing-critical section. These properties make it an attractive basis for a behavioral experiment library that prioritizes timing reliability and deployment simplicity.

The present article describes the design and capabilities of `goxpyriment`, discusses its timing architecture, and situates it relative to existing tools. Specifically, we highlight three main contributions of this framework: (1) zero-dependency deployment via single compiled binaries that simplify distribution across laboratory hardware; (2) native high-

precision timing achieved directly through the Go and SDL3 APIs without requiring external C/C++ helper engines; and (3) a linear, straightforward API that is highly suited for modern AI-assisted coding workflows.

Core Advantages of a Compiled Framework

The choice of Go as the implementation language for `goxpyriment` provides several critical advantages over the interpreted languages traditionally used in behavioral research.

Single-Binary Deployment (Zero Dependencies)

Go compiles program into a single statically linked executable with no external runtime dependencies. Crucially, the `go-sdl3` package embeds the pre-built SDL3 runtime libraries (SDL Developers, 2026) directly into the executable. The stimuli, experimental lists, fonts, ... can be embedded too. As a results, distributing an experiment requires copying just one file; there is no need to manage virtual environments, package versions, or interpreter installations across laboratory computers. This substantially reduces the operational overhead of running experiments on multiple machines or sharing materials with collaborators.

Explicit Garbage Collection Control

While the garbage collectors in languages like Python or JavaScript can introduce unpredictable pause times that corrupt timing, Go exposes a runtime API for suspending the garbage collector. `goxpyriment` uses this mechanism in its timing-critical loops, ensuring that stimulus onsets and offsets are not delayed by memory management tasks.

An AI-Friendly API

Go's design emphasizes simplicity and predictability. Because the language has few synonymous constructs and a strict type system, it is particularly well-suited for use with AI-powered coding assistants. The linear, straightforward API of `goxpyriment` allows researchers to leverage these assistants to bridge gaps in their programming experience, with the Go compiler providing a safety net by catching many common errors before the experiment is even run.

Timing Architecture and Precision

Hardware-Synchronized Presentation

`goxpyriment` achieves equivalent low-level control through the native SDL3 API (SDL Developers, 2026). `screen.Update()` calls SDL's `SDL_RenderPresent`, which blocks until the display's vertical retrace when VSYNC is enabled. Variable refresh rate (VRR) monitors are also supported, as described in the following section.

Display Compositor Bypass

A compositing window manager (compositor) intercepts every application frame and blends it with other on-screen elements before sending the result to the display. This introduces additional latency and jitter that are problematic for millisecond-accurate stimulus

presentation. The degree to which `goxpyriment` can avoid this overhead depends on the operating system.

On **Linux with X11**, SDL3 automatically sets the `_NET_WM_BYPASS_COMPOSITOR` window property when the application enters fullscreen mode. Compliant compositors (KWin, Mutter, Compton) respond by *unredirecting* the fullscreen window—routing its framebuffer directly to the display scan-out pipeline without compositing. The result is presentation latency below 1 ms, comparable to a compositor-free session. On **Linux with Wayland**, the compositor is always the intermediary for frame delivery, but modern compositors (KWin 5.21+, GNOME Mutter 44+) support *direct scan-out* for fullscreen applications, achieving similar low-latency behavior in practice. The best achievable timing on Linux is obtained by running without any display server: setting the environment variable `SDL_VIDEO_DRIVER=kmsdrm` before launching the experiment directs SDL3 to communicate with the Linux KMS/DRM subsystem directly, bypassing X11 and Wayland entirely. This configuration—typically used from a virtual terminal (`Ctrl+Alt+F2`)—is recommended for the most demanding timing requirements such as single-frame subliminal stimulation or high-frequency RSVP.

On **Windows**, SDL3's fullscreen mode creates an exclusive fullscreen surface that bypasses the Desktop Window Manager (DWM), again yielding frame-interval jitter typically below 0.3 ms (one standard deviation), comparable to Linux.

macOS is the exception. Apple's Metal rendering pipeline always delivers frames to the display through the WindowServer compositor, and no third-party application can bypass it—even in fullscreen. The WindowServer typically adds one frame of fixed latency (8–17 ms depending on refresh rate) and 2–5 ms of frame-interval jitter. This onset uncertainty — larger than many SOAs of experimental interest — makes macOS unsuitable for production data collection. Researchers should prefer Linux or Windows hardware for laboratory deployments of `goxpyriment`; macOS is adequate for pilot testing and paradigm development.

These platform differences can be quantified empirically with the `display` and `frames` sub-tests of the bundled `Timing-Tests` suite.

Variable Refresh Rate (VRR) Stimulus Presentation

Standard displays refresh at a fixed rate (e.g., 60 Hz), which quantises every stimulus duration to multiples of the frame period (16.67 ms at 60 Hz). A researcher wishing to present a stimulus for 10 ms, 17 ms, or 23 ms cannot do so on such a display without relying on temporal dithering or hardware frame-blending techniques that introduce their own artefacts. Variable Refresh Rate (VRR) technology—marketed as AMD FreeSync, NVIDIA G-Sync, or the VESA Adaptive-Sync standard—removes this constraint by allowing the display to hold each frame for an arbitrary duration and refresh only when instructed to do so by the GPU.

`goxpyriment` exposes VRR through a single API call: `exp.Screen.SetVSync(0)` disables the VSYNC block, so that every subsequent `SDL_RenderPresent` call returns immediately. The following pattern then achieves arbitrary stimulus durations with duration error typically below 0.5 ms (one standard deviation):

1. Draw the stimulus and call `screen.FlipNS()`, which presents the frame and records

`onsetNS` (an SDL nanosecond timestamp) immediately after `Present()` returns.

2. Busy-wait for the desired duration d: spin until `time.Now()` $\geq$ `onset` $+ d$. The last $\sim 500\,\mu s$ use a CPU spin loop (already used in the trigger timing routines) to eliminate OS scheduling jitter.

3. Draw the blank screen, call `FlipNS()` again to record `offsetNS`.

The actual software-controlled duration is (`offsetNS` − `onsetNS`)$/10^6\,ms$. On a VRR-capable display this equals the true on-screen duration plus a small, constant hardware pipeline latency (GPU scan-out and panel response, typically 1–5 ms) that can be calibrated once with a photodiode using the `frames` timing sub-test of the `Timing-Tests` suite.

Every VRR panel supports VRR only within a finite refresh-rate window (e.g., 48–144 Hz, corresponding to frame durations of 6.9–20.8 ms for a typical gaming monitor). Durations outside this window cause the panel to revert to fixed-rate behaviour, re-introducing quantisation. The `Timing-Tests` suite includes a dedicated `vrr` sub-test that sweeps target durations from 1 ms to a user-specified maximum in 1 ms steps and reports duration errors at each step, directly revealing the panel's VRR window in the output data.

On a display without VRR support, disabling VSYNC produces frame tearing. The `vrr` sub-test detects this automatically: on a non-VRR display the duration errors cluster at multiples of the frame period rather than being uniformly small, providing an unambiguous diagnostic.

High-Precision Streams

For paradigms with dense, high-speed stimulus sequences (e.g., Retinotopic stimulation), `goxpyriment` provides specialized stream functions like `PresentStreamOfImages`. These functions disable garbage collection, drain the event queue, and busy-wait at the sub-frame level to ensure that each stimulus onset is scheduled accurately. A `TimingLog` records the target and actual onsets for post-hoc verification, as well as the timing of keys pressed during the presentation of the stream.

Hardware-Level Response Timestamps

Response timing in `goxpyriment` leverages SDL3 hardware event timestamps. The `WaitKeysEventRT` method returns the `KeyboardEvent.Timestamp` field, which is populated by the OS input subsystem at hardware-interrupt time (nanoseconds since SDL initialization). By comparing this with the `Screen.FlipNS()` timestamp—which uses the same clock—reaction times can be measured with minimal jitter and no interpreter overhead. This unified timing regime is abstracted through the `ResponseDevice` interface, which explicitly signals whether a timestamp is "precise" (hardware-level) or "non-precise" (polling-based).

The Timing Verification Suite

The repository includes a dedicated timing verification suite (`tests/Timing-Tests/`) that allows researchers to characterize their hardware's performance. The suite includes tests for measuring frame-interval jitter, verifying single-frame

flash reliability via photodiode, and calculating audio-visual SOA latencies. Other tests measure hardware-buffer pipeline latency for audio and characterize the USB-serial jitter of trigger devices. Other tests could be implemented upon request by users to the first author. A special folder in goxpyriment's github repository, namely 'performance-reports', is meant for users to share the results of tests on their own hardware.

Lifecycle and Safety

Every experiment centers on an `Experiment` object that manages the window, renderer, and audio subsystems. Execution is typically wrapped in a `Run` function that provides a safety layer: if the participant presses the Escape key or closes the window, the library catches the resulting internal signal, safely flushes the data file to disk, and shuts down the SDL subsystems. This ensures that data is preserved even if a session is aborted prematurely.

Defining Trials and Data Logging

Experiments follow a hierarchical structure of blocks and trials directly inspired expyriment. Each trial is a collection of factors that can be randomized and logged. Data collection is handled by an `exp.Data` object, which writes to a CSV file with extensive metadata headers.

Pre-Experiment Setup Dialog

`goxpyriment` provides an optional graphical `GetParticipantInfo` dialog that opens before the main experiment window (See Fig. 1) It allows the experimenter to input subject IDs, select experimental conditions from discrete menus, and toggle settings like fullscreen mode. These values are automatically recorded in the output data file header.

Programs that use `GetParticipantInfo` also accept a `-headless` command-line flag. When present, the dialog is skipped entirely and field defaults (or values previously saved to the cache) are used without opening any window. This is convenient for scripted or automated runs (e.g. continuous integration checks or batch stimulus generation) where interactive input is not possible.

Comparison with Existing Tools

Table 1 summarizes key properties of `goxpyriment` alongside four widely used experiment frameworks.

Table 1. Comparison of selected experiment frameworks.

Property	goxpyriment	Expyriment	PsychoPy	jsPsych
Language	Go	Python	Python	JavaScript
Execution	Native binary	Interpreter	Interpreter	Browser
Deployment	Single binary	pip/conda env	Standalone app or pip	Node.js or CDN
GC control	Explicit disable	No	No	No
VSYNC sync	Yes (SDL3)	Yes (Pygame/SDL2)	Yes (Pyglet/PTB)	Limited (rAF)

Property	goxpyriment	Expyriment	PsychoPy	jsPsych
VRR / arbitrary durations	Yes (`SetVSync(0)`)	No	No	No
Hardware triggers	Yes	Yes	Yes	No
Response device API	Yes	Device-specific	Device-specific	N/A
Event timestamp source	SDL3 hardware	Python `time.time()`	iohub or Python clock	`performance.now()`
Stimulus set	Moderate	Moderate	Extensive	Moderate
GUI builder	No	No	Yes (Builder)	No
Online deployment	No	No	Partial (PsychoJS)	Yes
Reference	This paper	(Krause & Lindemann, 2014)	(Peirce et al., 2019)	(de Leeuw, 2015)

`goxpyriment` resembles Expyriment in its API design: a central experiment object, a `present()`-based stimulus API, a structured trial-loop model, and output to `.csv`-format files with metadata. The primary technical differences are the use of Go rather than Python and the use of SDL3 rather than the older Pygame/SDL2 stack. From the user point of view, goexpyriment adds 'streams' objects which, as their name indicate, allow to display streams of stimuli, withal tightly controlled timing.

PsychoPy offers a substantially larger stimulus set and a graphical Builder environment; `goxpyriment` has neither of these. However, PsychoPy's greater complexity also makes deployment and version management more challenging. jsPsych enables large-scale online data collection, which `goxpyriment` does not support. The MATLAB Psychophysics Toolbox (PTB) offers the most mature timing validation in the field; `goxpyriment` aims to achieve comparable precision through its native compiled architecture.

Example Experiments

The repository provides over forty examples demonstrating classic paradigms, including the Stroop task, Attentional Blink (RSVP), Masked Priming, Retinotopic mapping for fMRI, and adaptive threshold estimation using the QUEST algorithm. These examples serve as both functional templates and a gallery of the framework's capabilities, ranging from simple parity-judgment tasks to complex moving-dot attention studies.

Limitations and Future Directions

`goxpyriment` is currently at the beta-stage, that is, the API is stabilized and we are waiting for bug reports to address them.

The current library covers the stimuli most commonly needed in cognitive psychology but does not yet include complex 3D stimuli (which can nevertheless be programmed

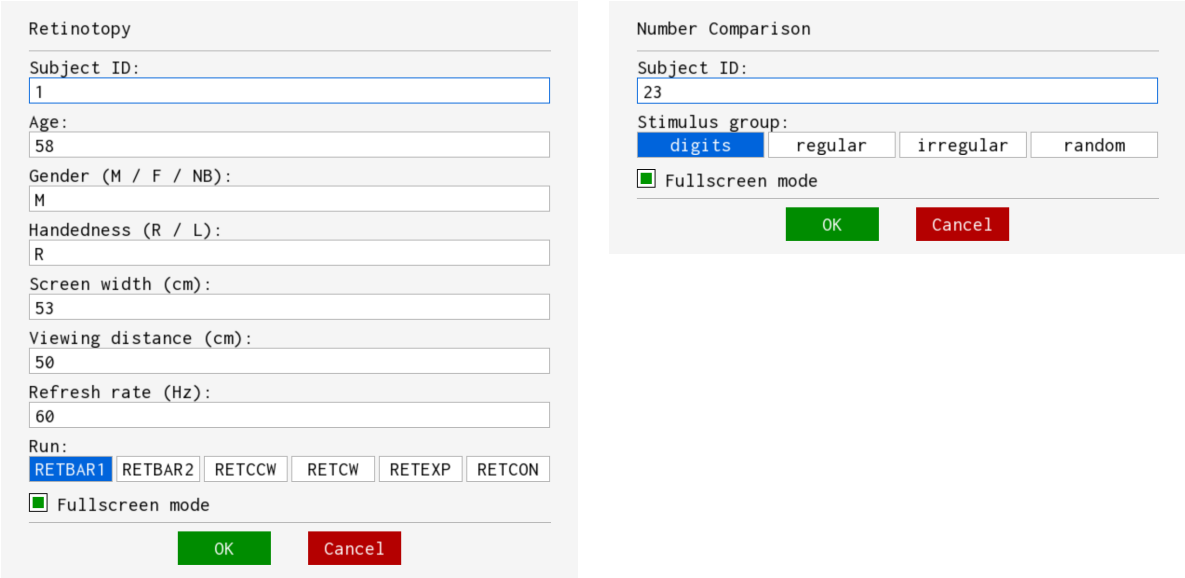


Figure 1

Examples of `GetParticipantInfo` *dialogs for the* Retinotopy *(left) and* Number-Comparison *(right) paradigms provided in the examples accompanying the library. These UIs makes it practical for the experimenter or participant to choose among a small fixed set of options without typing, which reduces setup errors when launching an experiment from a desktop shortcut or file manager. Note that which fields appear in the UI is configurable.*

directly using the underlying SDL api), nor has built-in eye-tracker support. We plan to add this in a future version.

A cautionary note about AI-assisted coding for psychologists

As mentioned above, goexpyriment is well-suited for AI assisted coding as its API and the provided examples provide strong guidance for AI coding agents prompted to program psychology experiments. The most recent experiments in the examples gallery were actually entirely generated by Claude or Gemini (see the section Authors' contributions below. Yet, even if these agents are extremely apt and quick at coding, human supervision is still clearly necessary as exemplified by Fig. 2.

Author Contributions

OB**Christophe Pallier**: Conceptualization, Software implementation, Documentation and Paper Writing, with assistance from **Claude Sonnet 4.6** and **Gemini 2.5**. Consistent with current editorial norms (Nature Portfolio, 2023; Springer Nature, 2023), Claude and Gemini are listed under Author Contributions rather than as a named author, because authorship requires the capacity to take accountability for the work. Christophe Pallier takes full responsibility for all content.

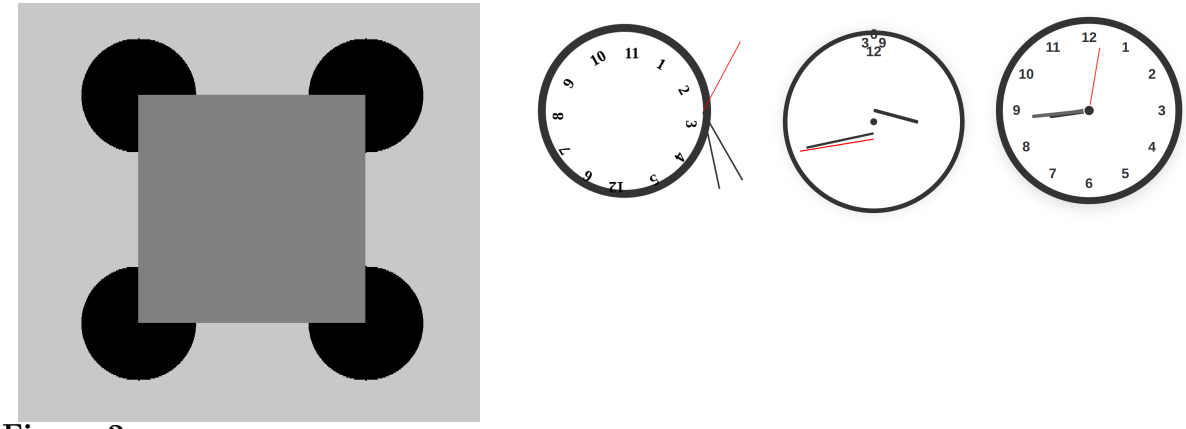


Figure 2

AI "blunders": *Left: Asked by the first author to write a program displaying Kanizsa's illusory rectangle, an AI produced the result above, explaining: "I have changed the Kanizsa example to make things robust and more visible [...] A darker gray rectangle is drawn at the center, clearly visible against the light-gray background.". Right: Output from the AI clock project where AIs are asked to write programs drawing a clock; note the correct response on the rigth (See https://clocks.brianmoore.com/ for more).*

Julie Bonnaire performed timing tests and redacted a report available in https://github.com/chrplr/goxpyriment/tree/mainperformance-reports

Marie-France Fourcade designed and built the photodiode system used to measure visual presentation delays in some tests.

Open Practices Statement

The source code of `goxpyriment` is available under the GNU General Public License v3 at https://github.com/chrplr/goxpyriment. All example experiments described in this article are included in the repository. No empirical data are reported in this article.

References

Anwyl-Irvine, A. L., Massonnié, J., Flitton, A., Kirkham, N., & Evershed, J. K. (2021). Gorilla in our midst: An online behavioral experiment builder. *Behavior Research Methods*, *52*(1), 388–407. https://doi.org/10.3758/s13428-019-01237-x

Brainard, D. H. (1997). The Psychophysics Toolbox. *Spatial Vision*, *10*(4), 433–436. https://doi.org/10.1163/156856897X00357

Bridges, D., Pitiot, A., MacAskill, M. R., & Peirce, J. W. (2020). The timing mega-study: Comparing a range of experiment generators, both lab-based and online. *PeerJ*, *8*, e9414. https://doi.org/10.7717/peerj.9414

de Leeuw, J. R. (2015). jsPsych: A JavaScript library for creating behavioral experiments in a Web browser. *Behavior Research Methods*, *47*(1), 1–12. https://doi.org/10.3758/s13428-014-0458-y

de Leeuw, J. R., Gilbert, R. A., & Luchterhandt, B. (2023). jsPsych: Enabling an open-source collaborative ecosystem of behavioral experiments. *Journal of Open Source Software*, *8*(85), 5351. https://doi.org/10.21105/joss.05351

Donovan, A. A. A., & Kernighan, B. W. (2016). *The Go programming language*. Addison-Wesley.

Henninger, F., Shevchenko, Y., Meijer, R. R., Inzlicht, M., & Hilbig, B. E. (2022). Lab.js: A free, open, online study builder. *Behavior Research Methods*, *54*(2), 556–573. https://doi.org/10.3758/s13428-019-01283-5

Kleiner, M., Brainard, D. H., & Pelli, D. (2007). What's new in Psychtoolbox-3? *Perception*, *36*(ECVP Abstract Supplement), 14.

Krause, F., & Lindemann, O. (2014). Expyriment: A Python library for cognitive and neuroscientific experiments. *Behavior Research Methods*, *46*(2), 416–428. https://doi.org/10.3758/s13428-013-0390-6

Nature Portfolio. (2023). Tools and technologies: Artificial intelligence.

Peirce, J. W., Gray, J. R., Simpson, S., MacAskill, M. R., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, *51*(1), 195–203. https://doi.org/10.3758/s13428-018-01193-y

SDL Developers. (2026). Simple DirectMedia Layer, sdl3 [Accessed: 2026-04-01].

Springer Nature. (2023). Artificial intelligence (AI) policy for authors.

appendix

Code Availability and Deployment

`goxpyriment` is available at https://github.com/chrplr/goxpyriment under the GPLv3 license. While the single-binary format simplifies distribution, users should be aware of modern operating system security features.

Windows: SmartScreen warning. When a user downloads a `.exe` file from the internet and tries to run it, Windows Defender SmartScreen may display a "Windows protected your PC" warning, because the executable has not been digitally signed. To proceed the user clicks **More info** and then **Run anyway**. This prompt appears only on first launch and only for files with the "downloaded from the internet" mark of origin; files copied from a USB drive or an internal network share bypass the check.

macOS: Gatekeeper. macOS includes a security system called **Gatekeeper** that checks whether applications have been signed with an Apple-issued certificate and submitted to Apple for notarization (a process that requires a paid Apple Developer account, $99/year). Applications distributed outside the Mac App Store that are not notarized receive a quarantine attribute when downloaded, which triggers Gatekeeper on first launch.

Two dialogs may appear, depending on how the user attempts to open the application.

1. **"App is damaged" dialog** (Figure 3). This alarming message appears when double-clicking an unsigned `.app` bundle. Despite the wording, the application has not been corrupted. **Workaround:** right-click (or Control-click) the `.app`, choose **Open** from the contextual menu, then click **Open** in the confirmation dialog. The exception is remembered; subsequent double-clicks launch the app normally.
2. **Privacy & Security panel** (Figure 4). After a blocked launch attempt, macOS records the blocked app in **System Settings → Privacy & Security**. Scrolling to the blocked-app entry and clicking **Open Anyway** (followed by the administrator password) has the same effect as the right-click method.

For plain command-line binaries (not `.app` bundles), the right-click workaround does not apply. Instead, the quarantine attribute can be removed with:

```
xattr -d com.apple.quarantine ./myexperiment
chmod +x ./myexperiment
./myexperiment
```

Linux. Linux binaries compiled with `go build` require no installation: copy the binary to the target machine, mark it executable (`chmod +x`), and run it. For broader distribution, the AppImage format (https://appimage.org) encapsulates a binary and its required shared libraries into a single portable file that runs on most Linux distributions without root access.

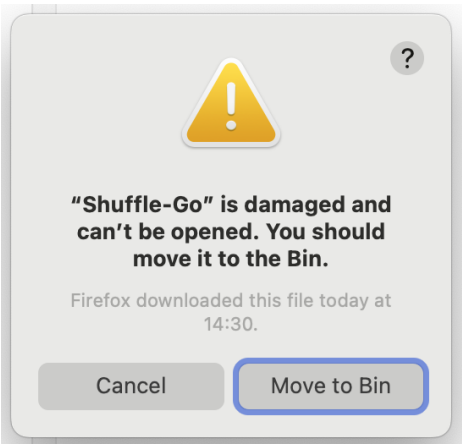
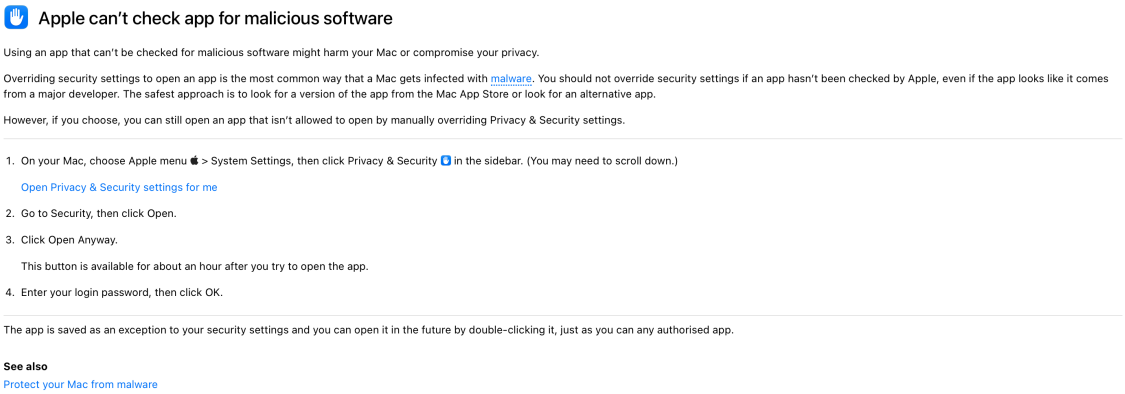


Figure 3

The macOS Gatekeeper "damaged" dialog that appears when double-clicking an unsigned `.app` *bundle downloaded from the internet. The application is not corrupted; the message reflects the absence of an Apple notarization certificate. Right-clicking the app and selecting* Open, *then confirming in the second dialog, bypasses the restriction permanently for that binary. Note that sometimes it is also necessary to run a 'xttr -d com.apple.quarantine myapp' command (see text)*



Apple can't check app for malicious software

Using an app that can't be checked for malicious software might harm your Mac or compromise your privacy.

Overriding security settings to open an app is the most common way that a Mac gets infected with malware. You should not override security settings if an app hasn't been checked by Apple, even if the app looks like it comes from a major developer. The safest approach is to look for a version of the app from the Mac App Store or look for an alternative app.

However, if you choose, you can still open an app that isn't allowed to open by manually overriding Privacy & Security settings.

1. On your Mac, choose Apple menu > System Settings, then click Privacy & Security in the sidebar. (You may need to scroll down.)

   Open Privacy & Security settings for me

2. Go to Security, then click Open.
3. Click Open Anyway.

   This button is available for about an hour after you try to open the app.

4. Enter your login password, then click OK.

The app is saved as an exception to your security settings and you can open it in the future by double-clicking it, just as you can any authorised app.

See also

Protect your Mac from malware

Figure 4

The Privacy & Security pane in macOS System Settings showing a blocked unsigned application. Clicking Open Anyway *and supplying the administrator password grants a permanent exception for that binary.*